

PROJECT 3: IPC: Inter-Process Communication

1 FOREWORD

Project 2 builds upon Project 1. This project contains two parts. In part 1, you will implement a proxy server, which takes in a GETFILE request from client side the same way as project 1 implementation and transfers it to the requested website using HTTP protocol to retrieve target files. In part 2, instead of making direct HTTP requests to the website, the proxy server will communicate with a local cache server. Your goal is to implement the cache server to answer the proxy requests and response with local cache files through shared memory segments.

2 PART ONE: PROXY SERVER

2.1 Project description

In this part, the job of a proxy server includes receiving file paths from clients, preparing HTTP requests, sending the requests using libcurl API, retrieving files from a web-based repository, and sending files back to clients.



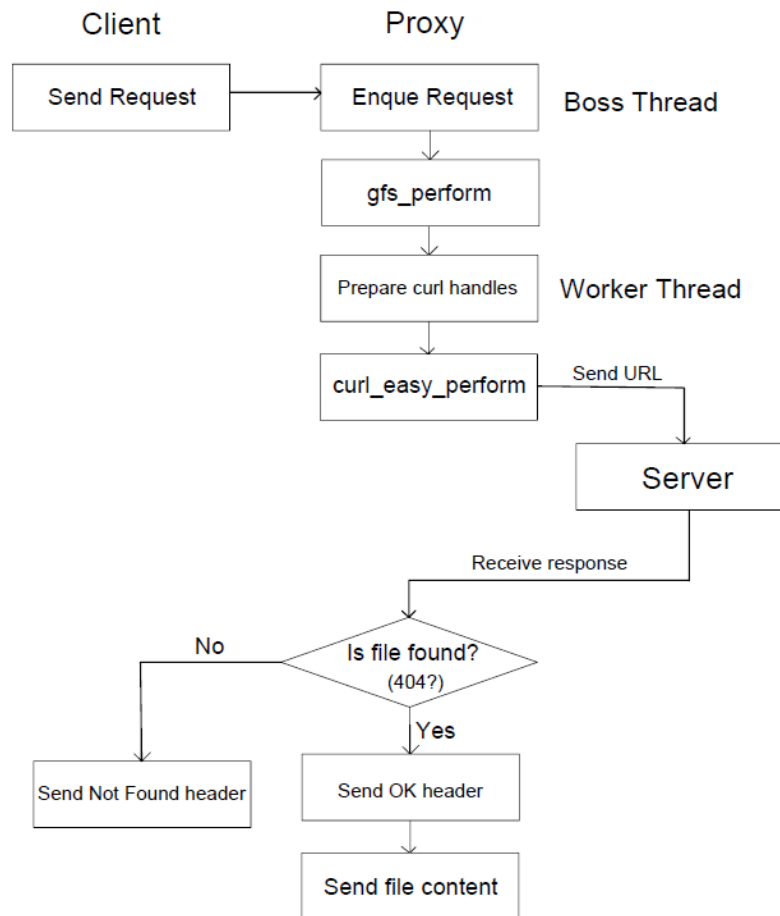
Credit: <https://github.gatech.edu/gios-spr-23/pr3>

The gfserver library takes care of the communication part with clients like project 1. The gfserver operates in a multithreaded manner as the boss-worker pattern. All you need to do is to implement a callback function named *"handle_with_url"* as one of the worker threads to fulfill the goal of data fetching from a website and send back responses via *gfs_sendheader* functions and data files via *gfs_send* function.

2.2 Flow of control

After the proxy server receives a request from a client, the task will eventually be passed into a work thread with task details. Then, the *handle_with_curl* function will wrap up the file path and server address into a full URL and employ the libcurl's *"easy"* API to set up a HTTP request and send it to the designated web-based repository. A *"retrieve_data"* function is passed into the *CURLOPT_WRITEFUNCTION* to reallocate memory as data is received from the repository.

It appends each new chunk of data to *CURLOPT_WRITEDATA* as a “response” buffer in struct “memory”, allowing the function to handle variable response sizes dynamically.



Once the online repository replies, the worker needs to check the response code. A 404 code returns a *GF_FILE_NOT_FOUND* header and no file is transferred. If data is found, a *GF_OK* header is sent by *gfs_sendheader* along with the retrieved data in “response” by *gfs_send*.

2.3 Implementation

The libcurl’s “easy” C interface is implemented to take care of HTTP requests and data retrieval.

The *curl_easy_init()* is called to initialize a curl easy handle. The *curl_easy_setopt()* is used to set options, such as the URL of the request, call back function for data retrieval and data buffer to store the file. The *curl_easy_perform()* is employed to send a request and use the call back function to save the data. The *curl_easy_cleanup()* frees the curl easy handle after the transfer is completed.

Struct “memory” contains a pointer “response” to the dynamically fetched content and its size.

2.4 Testing

Testing includes:

Functional Testing: Use `gfclient_download` to verify file retrieval accuracy and correct 404 error handling.

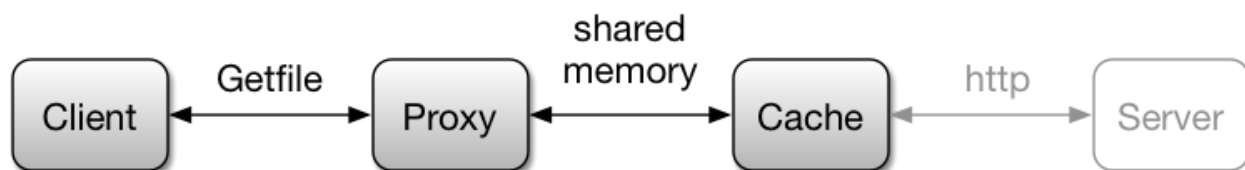
Performance Testing: Use `gfclient_measure` to assess multi-threading performance under various loads.

Edge Cases: Additional tests verify correct handling of large files, high-latency requests, and error cases.

3 PART TWO: CACHE SERVER

3.1 Project description

In this part, the proxy server minimizes external server requests by first checking for files in the cache. Thus, an additional cache server is introduced to manage the local cache database and communicate with the proxy server through shared memory-based inter-process communication (IPC). The cache and proxy communicate transfer commands via a command channel (message queues) and exchange file data through a data channel (shared memory). If the requested file is cached, it retrieves and serves the file; otherwise, it returns a `GF_FILE_NOT_FOUND` header.



Credit: <https://github.gatech.edu/gios-spr-23/pr3>

The design decision needs to be made for the choice of inter-process communication (IPC) of command channel and data channel. There are several IPC mechanisms available, such as pipes, message queues, sockets, and shared memories.

Pipes are simple to implement and work well for unidirectional data flow between a pair of processes, but they are less versatile for bidirectional communication, unlike message queues, which support asynchronous, multi-directional communication and prioritization of command

messages. Sockets are versatile and support networked IPC, enabling communication between processes on different hosts, which could add flexibility for a distributed system. However, sockets introduce additional protocol overhead (e.g., TCP/UDP setup and maintenance), which makes them comparatively slower for local IPC than shared memory and message queues.

On the other hand, message queues provide a reliable and ordered way to transfer commands asynchronously, supporting priority messages and ensuring messages aren't lost as they might be with signals. They are ideal for handling short control messages or commands, as they allow non-blocking send/receive operations, enabling efficient communication without requiring shared memory space.

Message queues are less efficient for transferring large data payloads, as each message is limited by the operating system's maximum queue size. In addition, message queues incur kernel overhead, as each message involves a system call, making them less suitable for high-throughput data transmission compared to shared memory. However, in this project, the command messages can be customized into a few variables with minimized size, making **POSIX message queues** an ideal choice for the **command channel** due to their stability, asynchronous communication, and prioritization features, which provide reliable control between the proxy and cache.

Shared memory is the fastest IPC mechanism for transferring large data payloads locally, as it enables direct access to a common memory segment without repeated system calls. However, it lacks built-in synchronization and requires additional mechanisms (e.g., semaphores) to manage concurrent access, adding complexity in multi-threaded contexts. **POSIX shared memory API** is leveraged for the **data channel**, allowing for efficient high-throughput data transfers with minimal latency, which is crucial for handling large files without introducing protocol overhead or system call bottlenecks. **Two semaphores (reader and writer)** are used for process synchronization and shared resource management to ensure data integrity.

This setup offers a robust and efficient architecture tailored to the project's needs, and provides hands-on experience with shared memory IPC and boss-worker multithreading patterns.

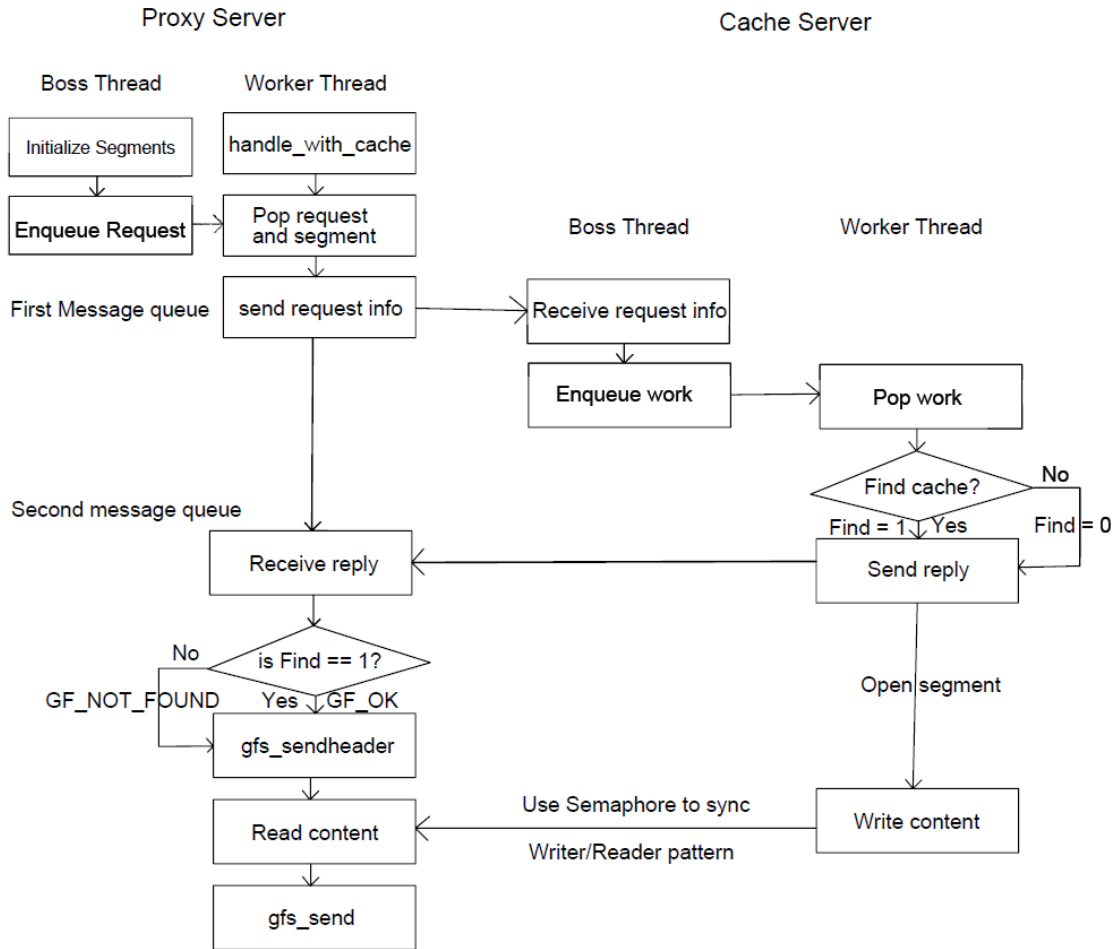
3.2 Flow of control

Figure below shows the overall system design. Proxy server initializes the shared memory segments pool, multiple threads, and a request queue along with mutex and condition variable, and then runs the gfservice to receive any upcoming file requests from clients. Proxy server is implemented in a boss-worker pattern. Boss thread sets up communication with clients and

distributes requests by enqueueing them in a steque. Worker thread picks up a request and initializes a message queue to pass the request along with the segment info to cache. Then, the worker opens a shared memory segment from the pool and waits for the response from the cache server.

Boss thread in the cache side is responsible for receiving requests from the proxy via the message queue, and enqueueing them into a work steque. Worker thread picks up a request, looks for the cache file, and then sets up a unique message queue to send back the lookup result. If a cache file is found, the worker opens the existing shared memory segment with the info passed in the request and reads the cache into the segment for data transferring.

In this design, there are two types of message queue used by proxy-cache communication. The first one is from worker thread at proxy side to boss thread at cache side for request transfer. This command channel is always opened throughout the entire run time to make sure the cache server receives requests from different worker threads, even different proxy servers. Then, the second type is established by the worker thread at cache side to transfer the lookup result to worker at proxy side. This message queue uses a unique name to ensure the specific worker thread at proxy that sends the request receives the response. The unique message queue is cleaned after each request is completed.



The data transfer design uses a **writer-reader model** with **two semaphores** to coordinate data transfer between a writer (cache) and reader (proxy). The writer reads chunks of data from a file and places each chunk in shared memory, where it waits for the reader to consume it. After each chunk is read and placed in shared memory, the writer signals the reader semaphore to wake up the reader thread and waits for the reader to complete the read and signal the writer semaphore. This approach ensures controlled, chunk-by-chunk data transfer, minimizing potential race conditions and synchronizing read and write operations effectively. Once all data is transferred, the writer and reader exchange final signals to confirm that the entire file has been processed before cleaning up the communication.

3.3 Implementation

The communication between proxy server and cache server is established using **POSIX message queue library** and the data transfer is accomplished by **POSIX shared memory API**.

There are two types of message queue initialized. One is employed by the worker thread in the proxy side to send a request to the boss thread in the cache side, while the second one is used by the worker thread in the cache side to send back a response to the request.

The message queue is initialized by *mq_open* function and used by *mq_send* function. The following information is included in the message (*proxy_msg_t*) for passing request details as well as info needed for data transferring. The name of this message queue is unique, because this one is only used for passing requests to the boss thread of the cache server, which is kept open through the entire run time and cleaned after the server is shut down.

```
typedef struct proxy_msg_t{  
  
    char file_path[BUFSIZE];  
  
    char server_name[BUFSIZE];  
  
    char shm_name[NAME];  
  
    char sem_w_name[NAME];  
  
    char sem_r_name[NAME];  
  
    size_t seg_size;  
  
} proxy_msg_t;
```

The second message queue is used by a cache worker to send a reply back to the proxy worker who sent the request. Thus, the name of the message queue must be unique to ensure the specific proxy worker will receive the response. The name is set up as a statics prefix combined with the name of the obtained shared memory segment to guarantee the uniqueness. The message itself (*cache_msg_t* below) is kept short to include only necessary information.

```
typedef struct cache_msg_t{  
  
    int find;  
  
    size_t file_size;  
  
} cache_msg_t;
```

The shared memory segments are initialized by the proxy boss and placed in a segment pool. When a proxy worker picks a work, it pops a segment from the pool as well for later use. On both sides, a while loop is implemented for transferring condition checks. The process begins by

initializing a *bytes_transferred* counter at zero and reading data in chunks defined by the smaller value between the remaining file size and the maximum *seg_size*. Since I used a writer-reader pattern for data transferring, two semaphores are necessary for synchronization between processes. This alternating semaphore pattern ensures that the writer and reader work in sync, with the reader reading each chunk and sending it to the client via *gfs_send* before signaling the writer to proceed with the next chunk. Before the segment is cleaned, writer and reader need to sync one more time to ensure transferring is completed.

When the server is interrupted or malfunctioned, all message queues and memory segments need to be cleaned as well as allocated memory for messages and work objects. The cleaning process is handled by *sig_handler* function on both sides.

3.4 Testing

Testing includes:

Functional Testing: Use *gfclient_download* with one thread to verify file retrieval accuracy.

Performance Testing: Use *gfclient_measure* to assess multi-threading performance under various loads.

Edge Cases: Additional tests written in python *ipcstress.py* to verify correct handling of large files, high-latency requests, variations on the parameters, and error cases.

4 REFERENCE

1. **Libcurl Documentation** - *libcurl: Multi-platform file transfer library*. Available online at: <https://curl.se/libcurl/>.
2. **POSIX Message Queues Documentation** - *POSIX.1-2001 Standard: Message Queue API*. Available at: https://man7.org/linux/man-pages/man7/mq_overview.7.html.
3. **POSIX Shared Memory Documentation** - *POSIX.1-2001 Standard: Shared Memory API*. Available at: https://man7.org/linux/man-pages/man7/shm_overview.7.html.
4. **Multithreading in C with POSIX Threads (Pthreads)** - *Guide to POSIX Threads*. Available at: <https://computing.llnl.gov/tutorials/pthreads/>.
5. **Testing: 6200 tools** - *Python test for various conditions*. Available at: <https://github.gatech.edu/cparaz3/6200-tools>.